Praktikum 7
Learning Vector Quantization

NAMA	:	Ihsanta Zaki Sanjaya
KELAS	:	2 D4 Teknik Komputer B
NRP	:	3223600058
DOSEN	:	Riyanto Sigit, ST, M.Kom, Ph.D
MATA KULIAH	:	Praktikum Sistem Cerdas
TANGGAL	:	Rabu, 16 April 2025

D4 TEKNIK KOMPUTER
POLITEKNIK ELEKTRONIKA NEGERI SURABAYA
2025

Praktikum 7
Membuat Program Aplikasi Learning Vector Quantization

A. Tujuan

1. Mahasiswa dapat memahami dan menjelaskan konsep Learning Vector Quantization
2. Mahasiswa dapat menjelaskan model Learning Vector Quantization
3. Mahasiswa dapat membuat aplikasi Learning Vector Quantization

Software yang diperlukan:

- Microsoft Visual C++
- PyCharm

B. Pendahuluan

1. Learning Vector Quantization

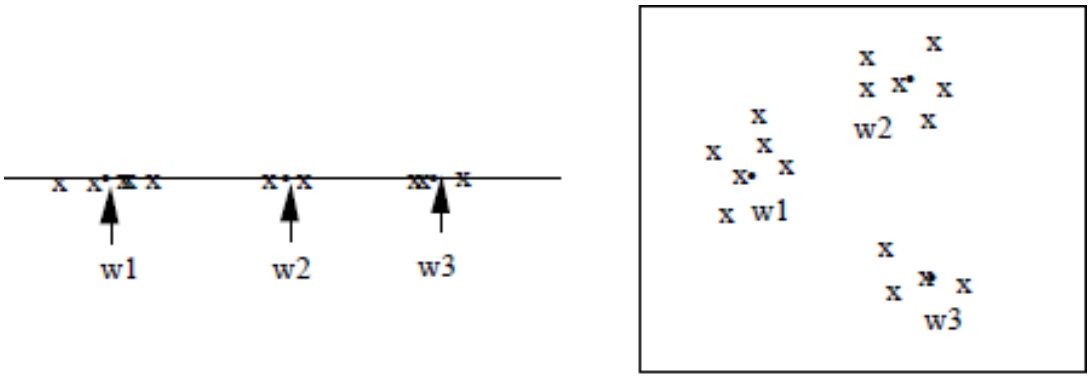
Learning Vector Quantization (LVQ) adalah salah satu metode klasifikasi dari Jaringan Syaraf Tiruan. LVQ bekerja dengan setiap unit output mempresentasikan sebuah kelas. Dengan kata lain LVQ adalah metode pengelompokan dimana target/kelasnya setiap kelompok/jumlah kelompok sudah ditentukan. Tujuan dari algoritma ini adalah untuk mendekati distribusi kelas vektor untuk meminimalkan kesalahan dalam pengklasifikasian.

Learning Vector Quantization (LVQ) adalah metode pembelajaran pada lapisan kompetitif yang terawasi. Suatu lapisan kompetitif akan secara otomatis belajar untuk mengklasifikasikan vektor-vektor input. Kelas-kelas yang didapatkan sebagai hasil dari lapisan kompetitif ini hanya tergantung pada jarak antara vektor-vektor input. Jika dua vektor input mendekati sama, maka lapisan kompetitif akan meletakkan kedua vektor input tersebut kedalam kelas yang sama.

Metode ini dibangun seperti menggunakan metode kohonen dengan perbedaan sistem ini adalah sistem supervised, yaitu melakukan klasifikasi vektor masukan berdasar vektor kuantisasinya. Perbedaan utama adalah cara melakukan pembaruan bobot vektornya.

2. Vector Quantization

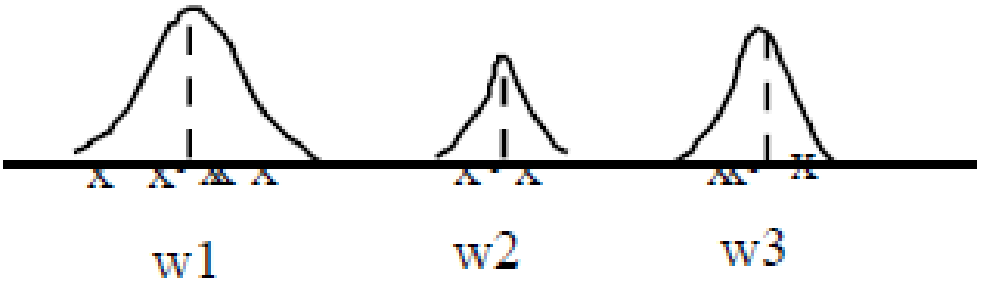
Dengan satu set vektor {x}, temukan satu set vektor perwakilan $\{w_m; 1 \leq m \leq M\}$ sehingga masing-masing x dikuantisasi ke dalam w_m tertentu.



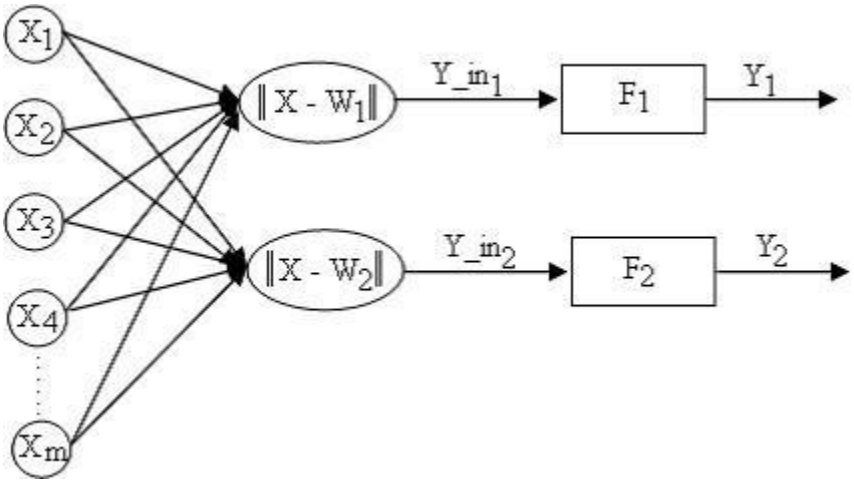
1-D (scaler) quantization

2-D vector quantization

VQ bergantung pada data $\{w_m\}$, cari rata-rata (centroid) dari distribusi kepadatan masing-masing cluster.



3. Arsitektur Learning Vector Quantization



Penjelasan gambar diatas adalah X adalah input, sedangkan W1 adalah vector bobot yang menghubungkan setiap neuron pada lapisan input ke neuron pertama pada lapisan output, sedangkan w2 adalah vector bobot yang menghubungkan setiap neuron pada lapisan input ke neuron kedua pada lapisan output.

LVQ sifatnya mencari jarak (euclidean) antara vektor-vektor input. Jika jaraknya pendek atau terkecil maka algoritma ini akan menempatkan data tersebut ke kelas atau target yang memiliki jaraknya terkecil.

4. Algoritma Learning Vector Quantization

1. Inisialisasi bobot wj dan derajat pembelajaran α(1)
2. Selama kondisi berhenti masih salah, kerjakan langkah 1-6
3. Untuk setiap vektor masukan pelatihan x kerjakan langkah 4-5
4. Temukan j sehingga |x-wj| minimum
5. Perbaharui wj sebagai berikut :

   Jika T = Cj maka

   $$Wj(t+1) = wj(t) + \alpha(t)[x(t) - wj(t)]$$ Jika

   T≠Cj maka

   $$Wj(t+1) = wj(t) - \alpha(t)[x(t) - wj(t)]$$

   Kurangi rerata pembelajaran α
6. Tes kondisi berhenti

5. Contoh Aplikasi Learning Vector Quantization

1. Speech Recognition
2. Robot Arm control
3. Industrial process control
4. automated synthesis of digital systems
5. channel equalization for telecommunication
6. image compression
7. radar classification of sea-ice
8. optimization problems
9. sentence understanding
10. classification of insect courtship songs

6. Contoh Implementasi Learning Vector Quantization

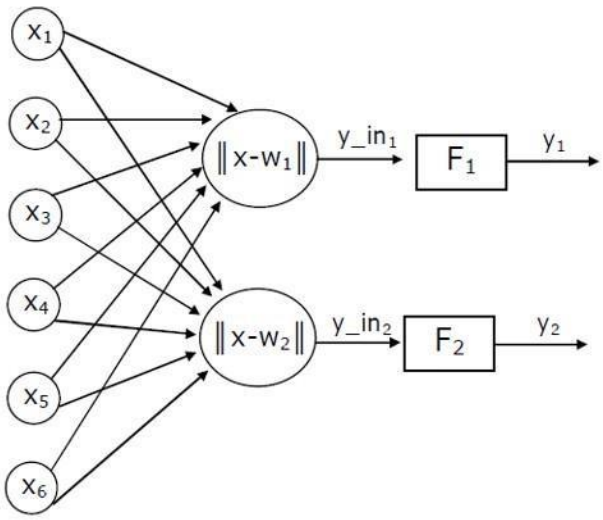
- Misalkan diketahui 10 input vektor dalam 2 kelas sebagai berikut:

```
 1. (1, 0, 0, 0, 1, 0)          1
 2. (0, 1, 1, 1, 1, 0)          2
 3. (0, 0, 1, 0, 0, 1)          1
 4. (0, 0, 1, 0, 1, 0)          1
 5. (0, 1, 0, 0, 0, 1)          1
 6. (1, 0, 1, 0, 1, 1)          1
 7. (0, 0, 1, 1, 0, 0)          2
 8. (0, 1, 0, 1, 0, 0)          2
 9. (1, 0, 0, 1, 0, 1)          2
10. (0, 1, 1, 1, 1, 1)          2
```

- Dua input pertama akan dijadikan sebagai inisialisasi bobot:

```
       Vektor (w)        Kelas
 1. (1, 0, 0, 0, 1, 0)      1
 2. (0, 1, 1, 1, 1, 0)      2
```

- Sedangkan 8 input sisanya akan digunakan sebagai data yang akan dilatih. Arsitektur jaringan syaraf pada seperti terlihat pada gambar.



- Sebagai nilai awal dipilih learning rate (α =0,05), dengan pengurangan sebesar 0,1*α; dan maksimum epoh (MaxEpoh=10)

  - Epoh ke-1:
  - Data ke-3: (0, 0, 1, 0, 0, 1)
  - Jarak pada:
    - bobot ke-1 =

$$\sqrt{(0-1)^2+(0-0)^2+(1-0)^2+(0-0)^2+(0-1)^2+(1-0)^2} = 2,0.$$

    - bobot ke-2 =

$$\sqrt{(0-1)^2+(0-1)^2+(1-1)^2+(0-1)^2+(0-1)^2+(1-0)^2} = 2,0.$$

  - Target data ke-3= 1
  - Bobot ke- 1 baru:

$$w_{11} = w_{11} + \alpha*(x_{11} - w_{11}) = 1 + 0,05*(0-1) = 0,9500;$$
$$w_{12} = w_{12} + \alpha*(x_{12} - w_{12}) = 0 + 0,05*(0-0) = 0,0000;$$
$$w_{13} = w_{13} + \alpha*(x_{13} - w_{13}) = 0 + 0,05*(1-0) = 0,0500;$$
$$w_{14} = w_{14} + \alpha*(x_{14} - w_{14}) = 0 + 0,05*(0-0) = 0,0000;$$
$$w_{15} = w_{15} + \alpha*(x_{15} - w_{15}) = 1 + 0,05*(0-1) = 0,9500;$$
$$w_{16} = w_{16} + \alpha*(x_{16} - w_{16}) = 0 + 0,05*(1-0) = 0,0500;$$

$$w_1 = (0,9500 \quad 0,0000 \quad 0,0500 \quad 0,0000 \quad 0,9500 \quad 0,0500)$$

- Data ke-4: (0, 0, 1, 0, 1, 0)
- Jarak pada:
  - bobot ke-1 =

$$= \sqrt{(0-0{,}95)^2 + (0-0)^2 + (1-0{,}05)^2 + (0-0)^2 + (1-0{,}95)^2 + (0-0{,}05)^2}$$
$$= 1{,}3454.$$

  - bobot ke-2 =

$$= \sqrt{(0-0)^2 + (0-1)^2 + (1-1)^2 + (0-1)^2 + (1-1)^2 + (0-0)^2}$$
$$= 1{,}4142.$$

- Jarak terkecil pada bobot ke-1
- Target data ke-4= 1
- Bobot ke- 1 baru:

$$w_{11} = w_{11} + \alpha*(x_{21} - w_{11}) = 0{,}95 + 0{,}05*(0-0{,}95) = 0{,}9025;$$
$$w_{12} = w_{12} + \alpha*(x_{22} - w_{12}) = 0{,}00 + 0{,}05*(0-0{,}00) = 0{,}0000;$$
$$w_{13} = w_{13} + \alpha*(x_{23} - w_{13}) = 0{,}05 + 0{,}05*(1-0{,}05) = 0{,}0975;$$
$$w_{14} = w_{14} + \alpha*(x_{24} - w_{14}) = 0{,}00 + 0{,}05*(0-0{,}00) = 0{,}0000;$$
$$w_{15} = w_{15} + \alpha*(x_{25} - w_{15}) = 0{,}95 + 0{,}05*(1-0{,}95) = 0{,}9525;$$
$$w_{16} = w_{16} + \alpha*(x_{26} - w_{16}) = 0{,}05 + 0{,}05*(0-0{,}05) = 0{,}0475;$$

- Jadi

$$w_1 = (0{,}9025 \quad 0{,}0000 \quad 0{,}0975 \quad 0{,}0000 \quad 0{,}9525 \quad 0{,}0475)$$

- Data ke-5: (0, 1, 0, 0, 0, 1)
- Jarak pada:
  - bobot ke-1 =

$$= \sqrt{(0-0{,}9025)^2 + (1-0)^2 + (0-0{,}0975)^2 + (0-0)^2 + (0-0{,}9525)^2 + (1-0{,}0475)^2}$$
$$= 1{,}9075$$

  - bobot ke-2 =

$$= \sqrt{(0-0)^2 + (1-1)^2 + (0-1)^2 + (0-1)^2 + (0-1)^2 + (1-0)^2}$$
$$= 2{,}000.$$

- Jarak terkecil pada bobot ke-1
- Target data ke-5= 1
- Bobot ke- 1 baru:

$$w_{11} = w_{11} + \alpha*(x_{31} - w_{11}) = 0{,}9025 + 0{,}05*(0-0{,}9025) = 0{,}8574;$$
$$w_{12} = w_{12} + \alpha*(x_{32} - w_{12}) = 0{,}0000 + 0{,}05*(0-0{,}0000) = 0{,}0500;$$
$$w_{13} = w_{13} + \alpha*(x_{33} - w_{13}) = 0{,}0975 + 0{,}05*(1-0{,}0975) = 0{,}0926;$$
$$w_{14} = w_{14} + \alpha*(x_{34} - w_{14}) = 0{,}0000 + 0{,}05*(0-0{,}0000) = 0{,}0000;$$
$$w_{15} = w_{15} + \alpha*(x_{35} - w_{15}) = 0{,}9525 + 0{,}05*(1-0{,}9525) = 0{,}9049;$$
$$w_{16} = w_{16} + \alpha*(x_{36} - w_{16}) = 0{,}0475 + 0{,}05*(0-0{,}0475) = 0{,}0951;$$

- Jadi

$$w_1 = (0{,}8574 \quad 0{,}0500 \quad 0{,}0926 \quad 0{,}0000 \quad 0{,}9049 \quad 0{,}0951)$$

- Data ke-6: (1, 0, 1, 0, 1, 1)
- Jarak pada:
  - bobot ke-1 =

$$= \sqrt{(1-0{,}8574)^2 + (0-0{,}0500)^2 + (1-0{,}0926)^2 + (0-0)^2 + (1-0{,}9049)^2 + (1-0{,}0951)^2}$$
$$= 1{,}2938$$

  - bobot ke-2 =

$$= \sqrt{(1-0)^2 + (0-1)^2 + (1-1)^2 + (0-1)^2 + (1-1)^2 + (1-0)^2}$$
$$= 2{,}000.$$

- Jarak terkecil pada bobot ke-1
- Target data ke-6= 1
- Bobot ke- 1 baru:

$$w_{11} = w_{11} + \alpha*(x_{41} - w_{11}) = 0{,}8574 + 0{,}05*(1-0{,}8574) = 0{,}8645;$$
$$w_{12} = w_{12} + \alpha*(x_{42} - w_{12}) = 0{,}0500 + 0{,}05*(0-0{,}0500) = 0{,}0475;$$
$$w_{13} = w_{13} + \alpha*(x_{43} - w_{13}) = 0{,}0926 + 0{,}05*(1-0{,}0926) = 0{,}1380;$$
$$w_{14} = w_{14} + \alpha*(x_{44} - w_{14}) = 0{,}0000 + 0{,}05*(0-0{,}0000) = 0{,}0000;$$
$$w_{15} = w_{15} + \alpha*(x_{45} - w_{15}) = 0{,}9049 + 0{,}05*(1-0{,}9049) = 0{,}9096;$$
$$w_{16} = w_{16} + \alpha*(x_{46} - w_{16}) = 0{,}0951 + 0{,}05*(1-0{,}0951) = 0{,}1404;$$

- Jadi

$$w_1 = (0{,}8645 \quad 0{,}0475 \quad 0{,}1380 \quad 0{,}0000 \quad 0{,}9096 \quad 0{,}1404)$$

- Data ke-7: (0, 0, 1, 1, 0, 0)
- Jarak pada:
  - bobot ke-1 =

$$= \sqrt{(0-0{,}8645)^2 + (0-0{,}0475)^2 + (1-0{,}1380)^2 + (1-0)^2 + (0-0{,}9096)^2 + (0-0{,}1404)^2}$$
$$= 1{,}8275$$

  - bobot ke-2 =

$$= \sqrt{(0-0)^2 + (0-1)^2 + (1-1)^2 + (1-1)^2 + (0-1)^2 + (0-0)^2}$$
$$= 1{,}4142.$$

- Jarak terkecil pada bobot ke-2
- Target data ke-7= 2
- Bobot ke- 2 baru:

$$w_{21} = w_{21} + \alpha*(x_{51} - w_{21}) = 0 + 0{,}05*(1-0) = 0{,}0000;$$
$$w_{22} = w_{22} + \alpha*(x_{52} - w_{22}) = 1 + 0{,}05*(0-1) = 0{,}9500;$$
$$w_{23} = w_{23} + \alpha*(x_{53} - w_{23}) = 1 + 0{,}05*(1-1) = 1{,}0000;$$
$$w_{24} = w_{24} + \alpha*(x_{54} - w_{24}) = 1 + 0{,}05*(0-1) = 1{,}0000;$$
$$w_{25} = w_{25} + \alpha*(x_{55} - w_{25}) = 1 + 0{,}05*(1-1) = 0{,}9500;$$
$$w_{26} = w_{26} + \alpha*(x_{56} - w_{26}) = 0 + 0{,}05*(1-0) = 0{,}0000;$$

- Jadi

$$w_2 = (0{,}0000 \quad 0{,}9500 \quad 1{,}0000 \quad 1{,}0000 \quad 0{,}9500 \quad 0{,}0000)$$

- Data ke-8: (0, 1, 0, 1, 0, 0)
- Jarak pada:
  - bobot ke-1 =

$$= \sqrt{(0-0{,}8645)^2 + (1-0{,}0475)^2 + (0-0{,}1380)^2 + (1-0)^2 + (0-0{,}9096)^2 + (0-0{,}1404)^2}$$
$$= 1{,}8764$$

  - bobot ke-2 =

$$= \sqrt{(0-0)^2 + (1-0{,}9500)^2 + (0-1)^2 + (1-1)^2 + (0-0{,}9500)^2 + (0-0)^2}$$
$$= 1{,}3802.$$

- Jarak terkecil pada bobot ke-2
- Target data ke-8= 2
- Bobot ke- 2 baru:

$$w_{21} = w_{21} + \alpha*(x_{61} - w_{21}) = 0{,}0000 + 0{,}05*(0-0{,}0000) = 0{,}0000;$$
$$w_{22} = w_{22} + \alpha*(x_{62} - w_{22}) = 0{,}9500 + 0{,}05*(1-0{,}9500) = 0{,}9525;$$
$$w_{23} = w_{23} + \alpha*(x_{63} - w_{23}) = 1{,}0000 + 0{,}05*(0-1{,}0000) = 0{,}9500;$$
$$w_{24} = w_{24} + \alpha*(x_{64} - w_{24}) = 1{,}0000 + 0{,}05*(1-1{,}0000) = 1{,}0000;$$
$$w_{25} = w_{25} + \alpha*(x_{65} - w_{25}) = 0{,}9500 + 0{,}05*(0-0{,}9500) = 0{,}9025;$$
$$w_{26} = w_{26} + \alpha*(x_{66} - w_{26}) = 0{,}0000 + 0{,}05*(0-0{,}0000) = 0{,}0000;$$

- Jadi

$$w_2 = (0{,}0000 \quad 0{,}9525 \quad 0{,}9500 \quad 1{,}0000 \quad 0{,}9025 \quad 0{,}0000)$$

- Data ke-9: (1, 0, 0, 1, 0, 1)
- Jarak pada:
  - bobot ke-1 =

$$= \sqrt{(1-0{,}8645)^2 + (0-0{,}0475)^2 + (0-0{,}1380)^2 + (1-0)^2 + (0-0{,}9096)^2 + (1-0{,}1404)^2}$$
$$= 1{,}6143$$

  - bobot ke-2 =

$$= \sqrt{(1-0)^2 + (0-0{,}9525)^2 + (0-0{,}9500)^2 + (1-1)^2 + (0-0{,}9025)^2 + (1-0)^2}$$
$$= 2{,}1504$$

- Jarak terkecil pada bobot ke-1
- Target data ke-9= 2
- Bobot ke- 1 baru:

$$w_{11} = w_{11} - \alpha*(x_{71} - w_{11}) = 0{,}8645 - 0{,}0266*(1-0{,}8645) = 0{,}8577;$$
$$w_{12} = w_{12} - \alpha*(x_{72} - w_{12}) = 0{,}0475 - 0{,}0266*(0-0{,}0475) = 0{,}0499;$$
$$w_{13} = w_{13} - \alpha*(x_{73} - w_{13}) = 0{,}1380 - 0{,}0266*(0-0{,}1380) = 0{,}1449;$$
$$w_{14} = w_{14} - \alpha*(x_{74} - w_{14}) = 0{,}0000 - 0{,}0266*(1-0{,}0000) = -0{,}0500;$$
$$w_{15} = w_{15} - \alpha*(x_{75} - w_{15}) = 0{,}9096 - 0{,}0266*(0-0{,}9096) = 0{,}9551;$$
$$w_{16} = w_{16} - \alpha*(x_{76} - w_{16}) = 0{,}1404 - 0{,}0266*(1-0{,}1404) = 0{,}0974;$$

- Jadi

$$w_1 = (0{,}8577 \quad 0{,}0499 \quad 0{,}1449 \quad -0{,}0500 \quad 0{,}9511 \quad 0{,}0974)$$

- Data ke-10: (0, 1, 1, 1, 1, 1)
- Jarak pada:
  - bobot ke-1 =

$$= \sqrt{(0-0{,}8577)^2 + (1-0{,}0499)^2 + (1-0{,}1449)^2 + (1+0{,}0500)^2 + (1-0{,}9551)^2 + (1-0{,}0974)^2}$$
$$= 2{,}0710$$

  - bobot ke-2 =

$$= \sqrt{(0-0)^2 + (1-0{,}9525)^2 + (1-0{,}9500)^2 + (1-1)^2 + (1-0{,}9025)^2 + (1-0)^2}$$
$$= 1{,}0028$$

- Jarak terkecil pada bobot ke-2
- Target data ke-10= 2
- Bobot ke- 2 baru:

$$w_{21} = w_{21} + \alpha*(x_{81} - w_{21}) = 0{,}0000 + 0{,}05*(0-0{,}0000) = 0{,}0000;$$
$$w_{22} = w_{22} + \alpha*(x_{82} - w_{22}) = 0{,}9525 + 0{,}05*(1-0{,}9525) = 0{,}9549;$$
$$w_{23} = w_{23} + \alpha*(x_{83} - w_{23}) = 0{,}9500 + 0{,}05*(0-0{,}9500) = 0{,}9525;$$
$$w_{24} = w_{24} + \alpha*(x_{84} - w_{24}) = 1{,}0000 + 0{,}05*(1-1{,}0000) = 1{,}0000;$$
$$w_{25} = w_{25} + \alpha*(x_{85} - w_{25}) = 0{,}9025 + 0{,}05*(0-0{,}9025) = 0{,}9074;$$
$$w_{26} = w_{26} + \alpha*(x_{86} - w_{26}) = 0{,}0000 + 0{,}05*(0-0{,}0000) = 0{,}0500;$$

- Jadi

$$w_2 = (0{,}0000 \quad 0{,}9549 \quad 0{,}9525 \quad 1{,}0000 \quad 0{,}9074 \quad 0{,}0500)$$
$$\alpha = \alpha - 0{,}1*\alpha = 0{,}05 - 0{,}1*0{,}05 = 0{,}045$$

- Proses tersebut diteruskan untuk epoh ke-2 sampai ke-10, untuk setiap data dengan menggunakan cara yang sama.
- Setelah mencapai epoh yang ke-10 diperoleh bobot akhir:

$$w_1 = (0{,}3727 \quad 0{,}2161 \quad 0{,}6347 \quad -0{,}2164 \quad 0{,}7981 \quad 0{,}4254)$$
$$w_2 = (0{,}0000 \quad 0{,}7969 \quad 0{,}7900 \quad 1{,}0000 \quad 0{,}5869 \quad 0{,}2171)$$

- Apabila kita ingin mensimulasikan input: (0, 1, 0, 1, 1, 0), maka kita cari terlebih dahulu jarak input tersebut dengan kedua bobot.
- Nomor dari bobot dengan jarak yang terpendek akan menjadi kelasnya.
- Jarak pada:
  - bobot ke-1 =

$$= \sqrt{(0-0{,}3727)^2 + (1-0{,}2161)^2 + (0-0{,}6347)^2 + (1+0{,}2164)^2 + (1-0{,}7981)^2 + (0-0{,}4254)^2}$$
$$= 1{,}6904$$

  - bobot ke-2 =

$$= \sqrt{(0-0)^2 + (1-0{,}7969)^2 + (0-0{,}7900)^2 + (1-1)^2 + (1-0{,}5869)^2 + (0-0{,}2171)^2}$$
$$= 0{,}9398$$

- Jarak terkecil pada bobot ke-2
- Sehingga input tersebut termasuk dalam kelas 2.

7. Contoh Program

```
#include "stdio.h"
#include <conio.h>
#include <math.h>

void main()
{
	int i, j, k, epoh = 0, maxIter = 10;
	float x[10][6] =
	{
		{1,0,0,0,1,0},
		{0,1,1,1,1,0},
		{0,0,1,0,0,1},
		{0,0,1,0,1,0},
		{0,1,0,0,0,1},
		{1,0,1,0,1,1},
		{0,0,1,1,0,0},
		{0,1,0,1,0,0},
		{1,0,0,1,0,1},
		{0,1,1,1,1,1}
	};
	int T[10] = { 1,2,1,1,1,1,2,2,2,2 };
	float w[2][6], jarak[2], alpha = 0.05f;
	//inisialisasi bobot
	jarak[0] = 0.0;
	jarak[1] = 0.0;
	for (i = 0; i < 6; i++)
	{
		w[0][i] = x[0][i];
		w[1][i] = x[1][i];
	}
	//traning
	for (i = 0; i < 10; i++)
	{
		for (j = 2; j < 10; j++)
		{
			jarak[0] = 0.0;
			jarak[1] = 0.0;
			for (k = 0; k < 6; k++) {
				jarak[0] = jarak[0] + (x[j][k] - w[0][k]) * (x[j][k] -
w[0][k]);
				jarak[1] = jarak[1] + (x[j][k] - w[1][k]) * (x[j][k] -
w[1][k]);
			}
			jarak[0] = sqrt(jarak[0]);
			jarak[1] = sqrt(jarak[1]);
			printf("jarak[0]:%f\n", jarak[0]);
				printf("jarak[1]:%f\n", jarak[1]);
```

```
if (jarak[0] <= jarak[1]) {
                            printf("jarak[0]\n");
                            if (T[j] == 1) {
                                    for (k = 0; k < 6; k++) {
                                            w[0][k] = w[0][k] + alpha * (x[j][k] -
w[0][k]);
                                            printf("w[0][%d]:%f\n", k, w[0][k]);
                                    }
                            }
                            else {
                                    for (k = 0; k < 6; k++) {
                                            w[0][k] = w[0][k] - alpha * (x[j][k] -
w[0][k]);
                                            printf("w[0][%d]:%f\n", k, w[0][k]);
                                    }
                            }
                    }
                    else {
                            printf("jarak[1]\n");
                            if (T[j] == 2) {
                                    for (k = 0; k < 6; k++) {
                                            w[1][k] = w[1][k] + alpha * (x[j][k] -
w[1][k]);
                                            printf("w[1][%d]:%f\n", k, w[1][k]);
                                    }
                            }
                            else {
                                    for (k = 0; k < 6; k++) {
                                            w[1][k] = w[1][k] - alpha * (x[j][k] -
w[0][k]);
                                            printf("w[1][%d]:%f\n", k, w[1][k]);
                                    }
                            }
                    }
            }
            alpha = alpha - 0.1 * alpha;
            epoh++;
            printf("Epoh:%d\n", epoh);
    }
```

```
        //running
        x[0][0] = 0;
        x[0][1] = 1;
        x[0][2] = 0;
        x[0][3] = 1;
        x[0][4] = 1;
        x[0][5] = 0;
        jarak[0] = 0.0;
        jarak[1] = 0.0;
        for (k = 0; k < 6; k++) {
               jarak[0] = jarak[0] + (x[0][k] - w[0][k]) * (x[0][k] - w[0][k]);
               jarak[1] = jarak[1] + (x[0][k] - w[1][k]) * (x[0][k] - w[1][k]);
        }
        jarak[0] = sqrt(jarak[0]);
        jarak[1] = sqrt(jarak[1]);
        printf("jarak[0]:%f\n", jarak[0]);
        printf("jarak[1]:%f\n", jarak[1]);
        if (jarak[0] <= jarak[1]) {
               printf("Kelas 1\n");
        }
        else {
               printf("Kelas 2\n");
        }
        }
```

8. Output

Microsoft Visual Studio Debug Console

```
jarak[1]
w[1][0]:0.000000
w[1][1]:0.792890
w[1][2]:0.785873
w[1][3]:1.000000
w[1][4]:0.578763
w[1][5]:0.201651
jarak[0]:1.779299
jarak[1]:1.794045
jarak[0]
w[0][0]:0.372660
w[0][1]:0.216126
w[0][2]:0.634700
w[0][3]:-0.216379
w[0][4]:0.798068
w[0][5]:0.425418
jarak[0]:1.654472
jarak[1]:0.950550
jarak[1]
w[1][0]:0.000000
w[1][1]:0.796902
w[1][2]:0.790020
w[1][3]:1.000000
w[1][4]:0.586923
w[1][5]:0.217116
Epoh:10
jarak[0]:1.690418
jarak[1]:0.939762
Kelas 2
```

C. Hasil Percobaan

Program:

```
#include "stdio.h"
#include <conio.h>
#include <math.h>

int main()
{
    int i, j, k, epoh = 0, maxIter = 10;
    float x[10][6] =
    {
        {1,0,0,0,1,0},
        {0,1,1,1,1,0},
        {0,0,1,0,0,1},
        {0,0,1,0,1,0},
        {0,1,0,0,0,1},
        {1,0,1,0,1,1},
        {0,0,1,1,0,0},
        {0,1,0,1,0,0},
        {1,0,0,1,0,1},
        {0,1,1,1,1,1}
    };
    int T[10] = { 1,2,1,1,1,1,2,2,2,2 };
    float w[2][6], jarak[2], alpha = 0.05f;
    //inisialisasi bobot
    jarak[0] = 0.0;
    jarak[1] = 0.0;
    for (i = 0; i < 6; i++)
    {
        w[0][i] = x[0][i];
        w[1][i] = x[1][i];
    }
    //traning
    for (i = 0; i < 10; i++)
    {
        for (j = 2; j < 10; j++)
        {
            jarak[0] = 0.0;
            jarak[1] = 0.0;
            for (k = 0; k < 6; k++) {
                jarak[0] = jarak[0] + (x[j][k] - w[0][k]) * (x[j][k] - w[0][k]);
                jarak[1] = jarak[1] + (x[j][k] - w[1][k]) * (x[j][k] - w[1][k]);
            }
            jarak[0] = sqrt(jarak[0]);
            jarak[1] = sqrt(jarak[1]);
            printf("jarak[0]:%f\n", jarak[0]);
            printf("jarak[1]:%f\n", jarak[1]);
            if (jarak[0] <= jarak[1]) {
                printf("jarak[0]\n");
```

```
                if (T[j] == 1) {
                    for (k = 0; k < 6; k++) {
                        w[0][k] = w[0][k] + alpha * (x[j][k] - w[0][k]);
                        printf("w[0][%d]:%f\n", k, w[0][k]);
                    }
                }
                else {
                    for (k = 0; k < 6; k++) {
                        w[0][k] = w[0][k] - alpha * (x[j][k] - w[0][k]);
                        printf("w[0][%d]:%f\n", k, w[0][k]);
                    }
                }
            }
            else {
                printf("jarak[1]\n");
                if (T[j] == 2) {
                    for (k = 0; k < 6; k++) {
                        w[1][k] = w[1][k] + alpha * (x[j][k] - w[1][k]);
                        printf("w[1][%d]:%f\n", k, w[1][k]);
                    }
                }
                else {
                    for (k = 0; k < 6; k++) {
                        w[1][k] = w[1][k] - alpha * (x[j][k] - w[0][k]);
                        printf("w[1][%d]:%f\n", k, w[1][k]);
                    }
                }
            }
        }
        alpha = alpha - 0.1 * alpha;
        epoh++;
        printf("Epoh:%d\n", epoh);
    }
    //running
    x[0][0] = 0;
    x[0][1] = 1;
    x[0][2] = 0;
    x[0][3] = 1;
    x[0][4] = 1;
    x[0][5] = 0;
    jarak[0] = 0.0;
    jarak[1] = 0.0;
    for (k = 0; k < 6; k++) {
        jarak[0] = jarak[0] + (x[0][k] - w[0][k]) * (x[0][k] - w[0][k]);
        jarak[1] = jarak[1] + (x[0][k] - w[1][k]) * (x[0][k] - w[1][k]);
    }
    jarak[0] = sqrt(jarak[0]);
    jarak[1] = sqrt(jarak[1]);
    printf("jarak[0]:%f\n", jarak[0]);
    printf("jarak[1]:%f\n", jarak[1]);
```

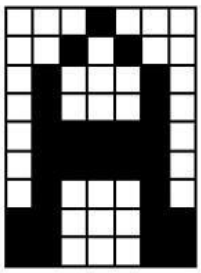
```
    if (jarak[0] <= jarak[1]) {
        printf("Kelas 1\n");
    }
    else {
        printf("Kelas 2\n");
    }
return 0;}
```

Hasil:

```
jarak[1]
w[1][0]:0.000000
w[1][1]:0.792890
w[1][2]:0.785873
w[1][3]:1.000000
w[1][4]:0.578763
w[1][5]:0.201651
jarak[0]:1.779299
jarak[1]:1.794045
jarak[0]
w[0][0]:0.372660
w[0][1]:0.216126
w[0][2]:0.634700
w[0][3]:-0.216379
w[0][4]:0.798068
w[0][5]:0.425418
jarak[0]:1.654472
jarak[1]:0.950550
jarak[1]
w[1][0]:0.000000
w[1][1]:0.796902
w[1][2]:0.790020
w[1][3]:1.000000
w[1][4]:0.586923
w[1][5]:0.217116
Epoh:10
jarak[0]:1.690418
jarak[1]:0.939762
Kelas 2
```

D. Tugas

- Akan dicoba untuk mengenali huruf A, B, atau H yang direpresentasikan dengan menggunakan kode 0 dan 1 pada matriks berukuran 9x7 seperti terlihat pada Gambar



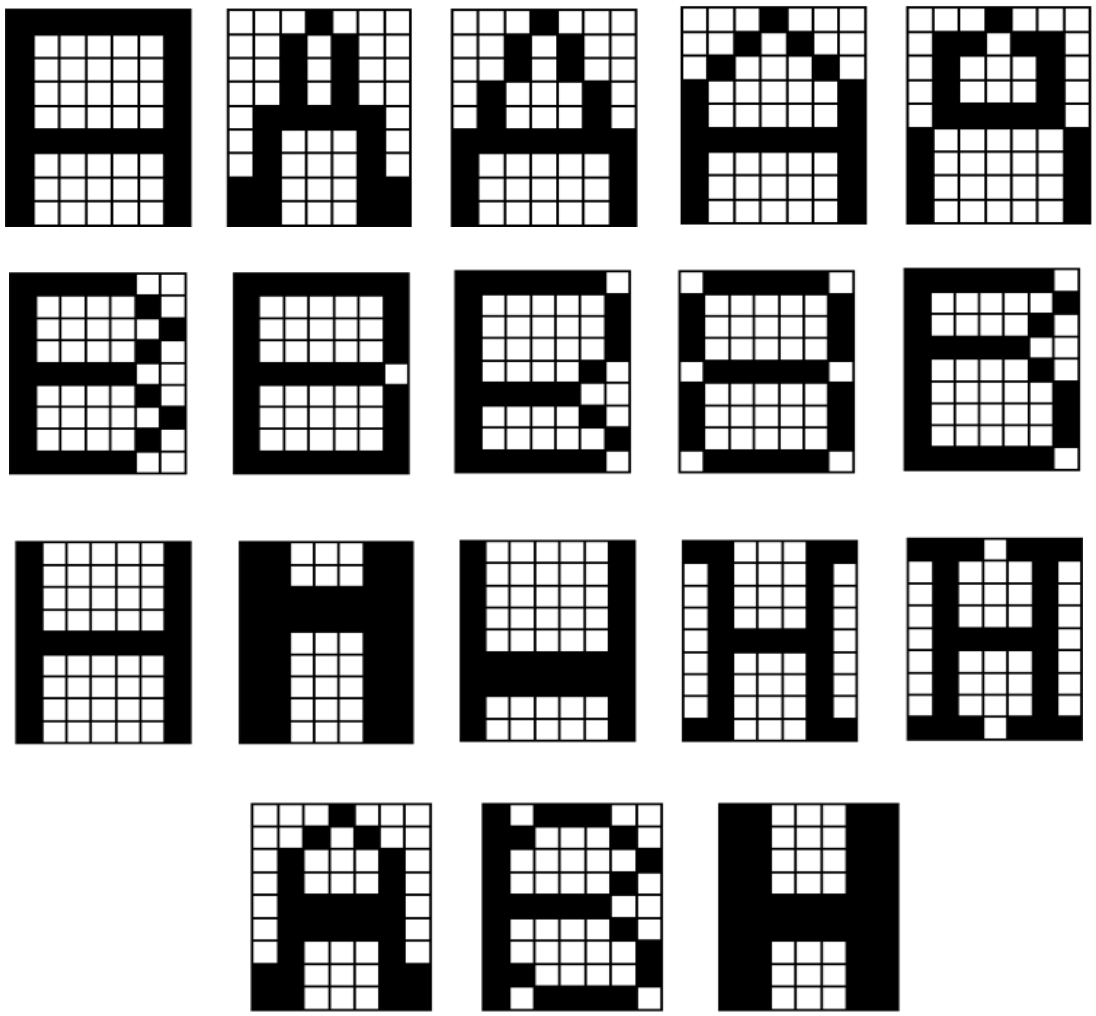
$$\begin{bmatrix} 0 & 0 & 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 1 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & 1 & 0 \\ 0 & 1 & 0 & 0 & 0 & 1 & 0 \\ 0 & 1 & 1 & 1 & 1 & 1 & 0 \\ 0 & 1 & 1 & 1 & 1 & 1 & 0 \\ 0 & 1 & 0 & 0 & 0 & 1 & 0 \\ 1 & 1 & 0 & 0 & 0 & 1 & 1 \\ 1 & 1 & 0 & 0 & 0 & 1 & 1 \end{bmatrix}$$

- Pada Gambar tersebut, kode 1 menunjukkan suatu kotak berwarna hitam, sedangkan kode 0 menunjukkan suatu kotak berwarna putih.

- Untuk mempermudah dalam implementasi, matriks 9x7 tersebut dibawa ke bentuk vektor 63 kolom. Pada Gambar tersebut vektor yang bersesuaian adalah:

0 0 0 1 0 0 0 0 0 1 0 1 0 0 0 1 0 0 0 1 0 0 1 0 0 0 1 0 0 1 1 1 1 1 0 0 1 1 1 1 1 0 0 1 0 0 0 1 0 1 1 0 0 0 1 1 1 1 0 0 0 1 1

- Misalkan ada 15 data yang akan dilatih, yaitu 5 data A, 5 data B, dan 5 data H, sebagai berikut



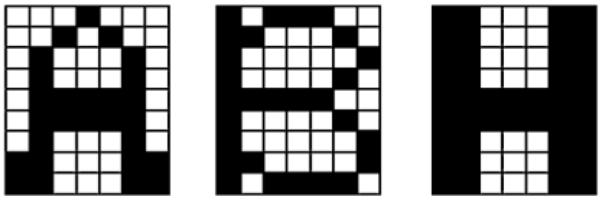
Matriks input yang digunakan dalam pelatihan memiliki bentuk sebagai berikut:

P=[1 1 1 1 1 1 1 1 0 0 0 0 0 1 1 0 0 0 0 0 1 1 0 0 0 0 0 1 1 0 0 0 0 0 1 1 1 1 1 1 1 1 1 0 0 0 0 0 1 1 0 0 0 0 0 1 1 0 0 0 0 0 1;
0 0 0 1 0 0 0 0 0 1 0 1 0 0 0 0 1 0 1 0 0 0 0 1 0 1 0 0 0 1 1 1 1 1 0 0 1 0 0 0 1 0 0 1 0 0 0 1 0 1 1 0 0 0 1 1 1 1 0 0 0 1 1;
0 0 0 1 0 0 0 0 0 1 0 1 0 0 0 0 1 0 1 0 0 0 1 0 0 0 1 0 0 0 1 0 0 1 0 0 0 1 0 1 1 1 1 1 1 1 1 1 0 0 0 0 0 1 1 0 0 0 0 0 1 1 0 0 0 0 0 1;
0 0 0 1 0 0 0 0 0 1 0 1 0 0 0 1 0 0 0 1 0 1 0 0 0 0 0 1 1 0 0 0 0 0 1 1 1 1 1 1 1 1 1 0 0 0 0 0 1 1 0 0 0 0 0 1 1 0 0 0 0 0 1;
0 0 0 1 0 0 0 0 1 1 0 1 1 0 0 1 0 0 0 1 0 0 1 0 0 0 1 0 0 1 1 1 1 1 0 1 0 0 0 0 0 1 1 0 0 0 0 0 1 1 0 0 0 0 0 1 1 0 0 0 0 0 1;
1 1 1 1 1 0 0 1 0 0 0 0 1 0 1 0 0 0 0 0 1 1 0 0 0 0 1 0 1 1 1 1 1 0 0 1 0 0 0 0 1 0 1 0 0 0 0 0 1 1 0 0 0 0 1 0 1 1 1 1 1 0 0;
1 1 1 1 1 1 1 1 0 0 0 0 0 1 1 0 0 0 0 0 1 1 0 0 0 0 0 1 1 1 1 1 1 1 0 1 0 0 0 0 0 1 1 0 0 0 0 0 1 1 0 0 0 0 0 1 1 1 1 1 1 1 1;
1 1 1 1 1 1 0 1 0 0 0 0 0 1 1 0 0 0 0 0 1 1 0 0 0 0 0 1 1 0 0 0 0 1 0 1 1 1 1 1 0 0 1 0 0 0 0 1 0 1 0 0 0 0 0 1 1 1 1 1 1 1 0;
1 1 1 1 1 1 0 1 0 0 0 0 0 1 1 0 0 0 0 0 1 1 0 0 0 0 0 1 1 0 0 0 0 1 0 1 1 1 1 1 0 1 0 0 0 0 0 1 1 0 0 0 0 0 1 1 0 0 0 0 0 1 0 1 1 1 1 1 0;
1 1 1 1 1 1 0 1 0 0 0 0 0 1 1 0 0 0 0 1 0 1 1 1 1 1 0 0 1 0 0 0 0 1 0 1 0 0 0 0 0 1 1 0 0 0 0 0 1 1 0 0 0 0 0 1 1 1 1 1 1 1 0;
1 0 0 0 0 0 1 1 0 0 0 0 0 1 1 0 0 0 0 0 1 1 0 0 0 0 0 1 1 1 1 1 1 1 1 1 0 0 0 0 0 1 1 0 0 0 0 0 1 1 0 0 0 0 0 1 1 0 0 0 0 0 1;
1 1 0 0 0 1 1 1 1 0 0 0 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 0 0 0 1 1 1 1 0 0 0 1 1 1 1 0 0 0 1 1 1 1 0 0 0 1 1 1 1 0 0 0 1 1;
1 0 0 0 0 0 1 1 0 0 0 0 0 1 1 0 0 0 0 0 1 1 0 0 0 0 0 1 1 0 0 0 0 0 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 0 0 0 0 0 1 1 0 0 0 0 0 1;
1 1 0 0 0 1 1 0 1 0 0 0 1 0 0 1 0 0 0 1 0 0 1 0 0 0 1 0 0 1 1 1 1 1 0 0 1 0 0 0 1 0 0 1 0 0 0 1 0 0 1 0 0 0 1 0 1 1 0 0 0 1 1;
1 1 1 0 1 1 1 0 1 0 0 0 1 0 0 1 0 0 0 1 0 0 1 0 0 0 1 0 0 1 1 1 1 1 0 0 1 0 0 0 1 0 0 1 0 0 0 1 0 0 1 0 0 0 1 0 1 1 1 0 1 1 1];

Dengan vektor output:

T=[1 1 1 1 1 2 2 2 2 2 3 3 3 3 3];

Angka 1 menunjukkan kelas A, angka 2 menunjukkan kelas B, dan angka 3 menunjukkan kelas H. Sedangkan untuk matriks bobot diperoleh dari transformasi pada gambar



w=[0 0 0 1 0 0 0 0 0 1 0 1 0 0 0 1 0 0 0 1 0 0 1 0 0 0 1 0 0 1 1 1 1 1 0 0 1 1 1 1 1 0 0 1 0 0 0 1 0 1 1 0 0 0 1 1 1 1 0 0 0 1 1;
1 0 1 1 1 0 0 1 1 0 0 0 1 0 1 0 0 0 0 0 1 1 0 0 0 0 1 0 1 0 1 1 1 0 0 1 0 0 0 0 1 0 1 0 0 0 0 0 1 1 1 0 0 0 0 1 1 0 1 1 1 1 0;
1 1 0 0 0 1 1 1 1 0 0 0 1 1 1 1 0 0 0 1 1 1 1 0 0 0 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 0 0 0 1 1 1 1 0 0 0 1 1 1 1 0 0 0 1 1];

Parameter pelatihan dimulai dengan nilai learning rate α = 0.05, yang akan dikurangi secara bertahap sebesar 10% dari α setiap epoh. Pelatihan dilakukan hingga maksimal 30 epoh.
Epoh ke-1:
Data ke-1:

P1 = (1 1 1 1 1 1 1 1 0 0 0 0 0 1 1 0 0 0 0 0 1 1 0 0 0 0 0 1 1 0 0 0 0 0 1 1 1 1 1 1 1 1 1 0 0 0 0 0 1 1 0 0 0 0 0 1 1 0 0 0 0 0 1);

Jarak terhadap bobot:

- Neuron 1 = 5,9161
- Neuron 2 = 4,7958
- Neuron 3 = 4,4721

Jarak terpendek ditemukan pada bobot ke-3
Target data ke-1= 1
Bobot ke- 3 baru (w3):
1,0000 1,0000 -0,0500 -0,0500 -0,0500 1,0000 1,0000 1,0000 ...
1,0500 0,0000 0,0000 0,0000 1,0500 1,0000 1,0000 1,0500 ...
0,0000 0,0000 0,0000 1,0500 1,0000 1,0000 1,0500 0,0000 ...
0,0000 0,0000 1,0500 1,0000 1,0000 1,0500 1,0500 1,0500 ...
1,0500 1,0500 1,0000 1,0000 1,0000 1,0000 1,0000 1,0000 ...
1,0000 1,0000 1,0000 1,0500 0,0000 0,0000 0,0000 1,0500 ...
1,0000 1,0000 1,0500 0,0000 0,0000 0,0000 1,0500 1,0000 ...
1,0000 1,0500 0,0000 0,0000 0,0000 1,0500 1,0000

α = 0,0450
α Data ke-2:

P2 = (0 0 0 1 0 0 0 0 0 1 0 1 0 0 0 0 1 0 1 0 0 0 0 1 0 1 0 0 0 1 1 1 1 1 0 0 1 0 0 0 1 0 0 1 0 0 0 1 0 1 1 0 0 0 1 1 1 1 0 0 0 1 1);

Jarak terhadap bobot:

- Neuron 1 = 3,3166
- Neuron 2 = 5,5678
- Neuron 3 = 5,7228

Jarak terpendek ditemukan pada bobot ke-1
Target data ke-2= 1
Bobot ke-1 baru (w1):
0,0000 0,0000 0,0000 1,0000 0,0000 0,0000 0,0000 0,0000 ...
0,0000 1,0000 0,0000 1,0000 0,0000 0,0000 0,0000 0,9550 ...
0,0450 0,0000 0,0450 0,9550 0,0000 0,0000 0,9550 0,0450 ...
0,0000 0,0450 0,9550 0,0000 0,0000 1,0000 1,0000 1,0000 ...
1,0000 1,0000 0,0000 0,0000 1,0000 0,9550 0,9550 0,9550 ...
1,0000 0,0000 0,0000 1,0000 0,0000 0,0000 0,0000 1,0000 ...
0,0000 1,0000 1,0000 0,0000 0,0000 0,0000 1,0000 1,0000 ...
1,0000 1,0000 0,0000 0,0000 0,0000 1,0000 1,0000
α = 0,0405

Langkah-langkah seperti di atas diterapkan untuk data ke-3 hingga ke-15. Setelah menyelesaikan seluruh data dalam satu epoh, proses dilanjutkan ke epoh berikutnya hingga mencapai epoh ke-30.

w1 =
-0,0297 -0,0297 -0,0140 1,0297 -0,0140 -0,0297 -0,0297 0,0000 ...
0,0113 1,0297 0,0000 1,0297 0,0113 0,0000 0,0000 0,9010 ...
0,0990 0,0000 0,0990 0,9010 0,0000 0,0443 0,9044 0,0513 ...
0,0000 0,0513 0,9044 0,0443 0,0443 0,9557 0,9080 0,9080 ...
0,9080 0,9557 0,0443 0,1330 0,9590 0,9375 0,9375 0,9375 ...
0,9590 0,1330 0,1330 0,8670 0,0000 0,0000 0,0000 0,8670 ...
0,1330 1,0297 0,8670 0,0000 0,0000 0,0000 0,8670 1,0297 ...
1,0000 0,8670 -0,0140 0,0000 -0,0140 0,8670 1,0000

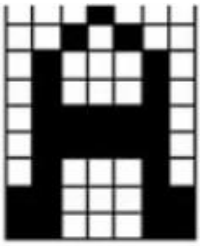
w2 =
1,0000 0,1358 1,0217 1,0217 1,0217 0,1014 -0,0005 1,0000 ...
0,8858 0,0000 0,0000 0,0000 0,9203 0,0797 1,0000 0,0000 ...
0,0000 0,0000 0,0000 0,0244 0,9756 1,0000 0,0244 0,0244 ...
0,0244 0,0244 0,9203 0,0553 0,9733 0,0712 0,9570 0,9570 ...
0,9570 0,0903 -0,0322 1,0000 0,0185 0,0185 0,0185 0,0185 ...
0,9097 0,0506 1,0000 0,0000 0,0000 0,0000 0,0000 0,0291 ...
0,9709 1,0000 0,8858 0,0000 0,0000 0,0000 0,0345 0,9655 ...
0,9733 0,1464 1,0322 1,0322 1,0322 0,9978 -0,0005

w3=
1,0000 0,9823 -0,0504 -0,0504 -0,0504 0,9823 1,0000 1,0000 ...
1,0327 0,0000 0,0000 0,0000 1,0327 1,0000 1,0000 1,0327 ...
0,0194 0,0194 0,0194 1,0327 1,0000 1,0000 1,0327 0,0194 ...
0,0194 0,0194 1,0327 1,0000 1,0000 1,0327 1,0132 1,0132 ...
1,0132 1,0327 1,0000 1,0000 1,0000 0,9806 0,9806 0,9806 ...
1,0000 1,0000 1,0000 1,0504 0,0177 0,0177 0,0177 1,0504 ...
1,0000 1,0000 1,0327 0,0000 0,0000 0,0000 1,0327 1,0000 ...
1,0000 1,0327 0,0000 0,0000 0,0000 1,0327 1,0000

Dilakukan testing input:

X = (0 0 0 1 0 0 0 0 0 0 1 0 0 0 0 0 1 0 1 0 0 0 0 1 0 1 0 0 0 1 0 0 0 1 0 0 1 0 0 0 1 0 1 1 1 1 1 1 1 1 0 0 0 0 0 1 1 0 0 0 0 0 1);

Seperti gambar dibawah



Langkah pertama adalah menghitung jarak antara input dengan masing-masing bobot. Input akan diklasifikasikan ke dalam kelas yang memiliki bobot dengan jarak terdekat.
Hasil perhitungan jaraknya:
Jarak pada bobot 1 = 0,0693
Jarak pada bobot 2 = 1,7655
Jarak pada bobot 3 = 2,2446

Karena jarak terkecil terdapat pada bobot pertama, maka input tersebut dikategorikan ke dalam kelas A.

Program:

```
#include <iostream>
#include <cmath>
#include <vector>
using namespace std;

const int JML_DATA = 15;
const int JML_FITUR = 63;
const int JML_KELAS = 3;
const int MAX_EPOH = 10;

float data[JML_DATA][JML_FITUR] = {
    // 5 huruf A
    {0,0,0,1,0,0,0, 0,0,1,0,0,1,0, 0,1,0,0,0,1,0, 0,1,0,0,0,1,0, 0,1,1,1,1,1,0,
0,1,0,0,0,1,0, 0,1,0,0,0,1,0, 1,0,0,0,0,0,1, 1,0,0,0,0,0,1},
    {0,0,0,1,0,0,0, 0,0,1,0,0,1,0, 0,1,0,0,0,1,0, 0,1,0,0,0,1,0, 0,1,1,1,1,1,0,
0,1,0,0,0,1,0, 0,1,0,0,0,1,0, 1,0,0,0,0,0,1, 1,0,0,0,0,0,1},
```

```
    {0,0,0,1,0,0,0, 0,0,1,0,0,1,0, 0,1,0,0,0,1,0, 0,1,0,0,0,1,0, 0,1,1,1,1,1,0,
0,1,0,0,0,1,0, 0,1,0,0,0,1,0, 1,0,0,0,0,0,1, 1,0,0,0,0,0,1},
    {0,0,0,1,0,0,0, 0,0,1,0,0,1,0, 0,1,0,0,0,1,0, 0,1,0,0,0,1,0, 0,1,1,1,1,1,0,
0,1,0,0,0,1,0, 0,1,0,0,0,1,0, 1,0,0,0,0,0,1, 1,0,0,0,0,0,1},
    {0,0,0,1,0,0,0, 0,0,1,0,0,1,0, 0,1,0,0,0,1,0, 0,1,0,0,0,1,0, 0,1,1,1,1,1,0,
0,1,0,0,0,1,0, 0,1,0,0,0,1,0, 1,0,0,0,0,0,1, 1,0,0,0,0,0,1},
    // 5 huruf B
    {1,1,1,1,1,1,0, 1,0,0,0,0,0,1, 1,0,0,0,0,0,1, 1,1,1,1,1,1,0, 1,0,0,0,0,0,1,
1,0,0,0,0,0,1, 1,0,0,0,0,0,1, 1,1,1,1,1,1,0, 0,0,0,0,0,0,0},
    {1,1,1,1,1,1,0, 1,0,0,0,0,0,1, 1,0,0,0,0,0,1, 1,1,1,1,1,1,0, 1,0,0,0,0,0,1,
1,0,0,0,0,0,1, 1,0,0,0,0,0,1, 1,1,1,1,1,1,0, 0,0,0,0,0,0,0},
    {1,1,1,1,1,1,0, 1,0,0,0,0,0,1, 1,0,0,0,0,0,1, 1,1,1,1,1,1,0, 1,0,0,0,0,0,1,
1,0,0,0,0,0,1, 1,0,0,0,0,0,1, 1,1,1,1,1,1,0, 0,0,0,0,0,0,0},
    {1,1,1,1,1,1,0, 1,0,0,0,0,0,1, 1,0,0,0,0,0,1, 1,1,1,1,1,1,0, 1,0,0,0,0,0,1,
1,0,0,0,0,0,1, 1,0,0,0,0,0,1, 1,1,1,1,1,1,0, 0,0,0,0,0,0,0},
    {1,1,1,1,1,1,0, 1,0,0,0,0,0,1, 1,0,0,0,0,0,1, 1,1,1,1,1,1,0, 1,0,0,0,0,0,1,
1,0,0,0,0,0,1, 1,0,0,0,0,0,1, 1,1,1,1,1,1,0, 0,0,0,0,0,0,0},
    // 5 huruf H
    {1,0,0,0,0,0,1, 1,0,0,0,0,0,1, 1,0,0,0,0,0,1, 1,1,1,1,1,1,1, 1,0,0,0,0,0,1,
1,0,0,0,0,0,1, 1,0,0,0,0,0,1, 1,0,0,0,0,0,1, 0,0,0,0,0,0,0},
    {1,0,0,0,0,0,1, 1,0,0,0,0,0,1, 1,0,0,0,0,0,1, 1,1,1,1,1,1,1, 1,0,0,0,0,0,1,
1,0,0,0,0,0,1, 1,0,0,0,0,0,1, 1,0,0,0,0,0,1, 0,0,0,0,0,0,0},
    {1,0,0,0,0,0,1, 1,0,0,0,0,0,1, 1,0,0,0,0,0,1, 1,1,1,1,1,1,1, 1,0,0,0,0,0,1,
1,0,0,0,0,0,1, 1,0,0,0,0,0,1, 1,0,0,0,0,0,1, 0,0,0,0,0,0,0},
    {1,0,0,0,0,0,1, 1,0,0,0,0,0,1, 1,0,0,0,0,0,1, 1,1,1,1,1,1,1, 1,0,0,0,0,0,1,
1,0,0,0,0,0,1, 1,0,0,0,0,0,1, 1,0,0,0,0,0,1, 0,0,0,0,0,0,0},
    {1,0,0,0,0,0,1, 1,0,0,0,0,0,1, 1,0,0,0,0,0,1, 1,1,1,1,1,1,1, 1,0,0,0,0,0,1,
1,0,0,0,0,0,1, 1,0,0,0,0,0,1, 1,0,0,0,0,0,1, 0,0,0,0,0,0,0}
};

int label[JML_DATA] = {
    0,0,0,0,0, // A
    1,1,1,1,1, // B
    2,2,2,2,2  // H
};

float w[JML_KELAS][JML_FITUR];
float alpha = 0.1f;

float euclideanDistance(float *x, float *w) {
    float sum = 0;
    for (int i = 0; i < JML_FITUR; i++) {
        sum += (x[i] - w[i]) * (x[i] - w[i]);
    }
    return sqrt(sum);
}

void trainLVQ() {
    // Inisialisasi bobot awal dari masing-masing kelas
```

```
    for (int i = 0; i < JML_FITUR; i++) {
        w[0][i] = data[0][i];  // A
        w[1][i] = data[5][i];  // B
        w[2][i] = data[10][i]; // H
    }

    for (int epoh = 0; epoh < MAX_EPOH; epoh++) {
        for (int i = 0; i < JML_DATA; i++) {
            float minDist = 1e9;
            int closest = -1;

            for (int c = 0; c < JML_KELAS; c++) {
                float d = euclideanDistance(data[i], w[c]);
                if (d < minDist) {
                    minDist = d;
                    closest = c;
                }
            }

            // Update bobot
            for (int j = 0; j < JML_FITUR; j++) {
                if (closest == label[i]) {
                    w[closest][j] += alpha * (data[i][j] - w[closest][j]);
                } else {
                    w[closest][j] -= alpha * (data[i][j] - w[closest][j]);
                }
            }
        }
        alpha *= 0.9f;
        cout << "Epoh " << epoh + 1 << " selesai.\n";
    }
}

void test(float testData[JML_FITUR]) {
    float minDist = 1e9;
    int kelas = -1;
    for (int c = 0; c < JML_KELAS; c++) {
        float d = euclideanDistance(testData, w[c]);
        if (d < minDist) {
            minDist = d;
            kelas = c;
        }
    }

    cout << "Data dikenali sebagai huruf: ";
    if (kelas == 0) cout << "A\n";
    else if (kelas == 1) cout << "B\n";
    else cout << "H\n";
}
```

```
int main() {
    trainLVQ();

    // Contoh pengujian data baru (huruf A)
    float newData[JML_FITUR] = {
        0,0,0,1,0,0,0, 0,0,1,0,0,1,0, 0,1,0,0,0,1,0, 0,1,0,0,0,1,0,
        0,1,1,1,1,1,0, 0,1,0,0,0,1,0, 0,1,0,0,0,1,0, 1,0,0,0,0,0,1,
        1,0,0,0,0,0,1
    };
    test(newData);

    return 0;
}
```

Hasil:

```
Epoh 1 selesai.
Epoh 2 selesai.
Epoh 3 selesai.
Epoh 4 selesai.
Epoh 5 selesai.
Epoh 6 selesai.
Epoh 7 selesai.
Epoh 8 selesai.
Epoh 9 selesai.
Epoh 10 selesai.
Data dikenali sebagai huruf: A
```

E. Analisa

Melalui rangkaian percobaan pada praktikum ini, telah dilakukan implementasi algoritma Learning Vector Quantization (LVQ) sebagai pendekatan dalam klasifikasi berbasis pembelajaran terawasi. LVQ bekerja dengan membentuk representasi vektor bobot (weight vector) untuk setiap kelas dan memperbaruinya secara iteratif berdasarkan data pelatihan. Inti dari algoritma ini adalah menghitung jarak antara sampel data dan bobot menggunakan Euclidean distance, lalu menyesuaikan bobot agar semakin mendekati pola data yang sesuai atau menjauhinya jika kelas berbeda.

Percobaan pertama berfokus pada klasifikasi dua kelas dengan menggunakan sepuluh data latih yang memiliki enam fitur biner. Dua bobot awal diambil dari dua sampel pertama. Program kemudian memperbarui bobot berdasarkan kecocokan antara kelas sampel dan bobot terdekat. Jika sesuai, bobot ditarik mendekati data; jika tidak, bobot didorong menjauh. Proses ini diulang selama beberapa iterasi (epoch), dengan laju pembelajaran (alpha) yang dikurangi secara bertahap untuk menjaga kestabilan perubahan bobot. Eksperimen lanjutan memperluas pendekatan LVQ untuk klasifikasi pola huruf A, B, dan H. Di sini, masing-masing huruf direpresentasikan sebagai citra biner berukuran 9x7 yang diubah menjadi array satu dimensi. Bobot awal diinisialisasi dari contoh masing-masing huruf, dan pembaruan dilakukan dengan prinsip yang sama. Setelah pelatihan, model diuji menggunakan data huruf baru yang tidak ada dalam pelatihan, dan mampu mengklasifikasikannya dengan benar.

Kedua pendekatan menunjukkan bahwa LVQ mampu mengenali pola dan membentuk representasi bobot yang efektif untuk klasifikasi. Faktor keberhasilan terletak pada:

- Pemilihan bobot awal yang representatif
- Mekanisme pembaruan bobot berbasis jarak dan kelas target
- Pengaturan learning rate yang menurun secara bertahap
- Penggunaan metrik Euclidean yang konsisten dalam mengukur kesamaan

F. Kesimpulan

Algoritma LVQ terbukti efektif dalam melakukan klasifikasi berbasis fitur dengan memanfaatkan representasi bobot yang diperbarui secara iteratif, mampu mengenali pola serta melakukan generalisasi terhadap data baru, sehingga cocok digunakan dalam berbagai aplikasi pengenalan pola seperti klasifikasi citra dan karakter.